

A Structured, Adjustable-View Medium for Scientific Code

Design Whitepaper — Requirements Distillation

Paul Agron

October 1, 2026 • Working draft

Abstract

Today we write and read programs as flat text — one long string of characters per file. This document proposes an environment for scientific computing in which a program is stored not as text but as *structure* (essentially, the parsed form of the code), and in which the way that structure is *shown on screen* is something the reader can adjust freely, moment to moment, without changing the program at all. A reader can hide whole categories of clutter (say, all logging), or display a piece of mathematics the way it appears in a textbook (a matrix as a grid, a fraction stacked) rather than squashed onto one line. Crucially, none of these adjustments alter the program; they change only how it is drawn. This paper explains the idea from first principles, states the design rules that keep it honest, lists the concrete features and the limits placed on each, and records both the decisions deliberately postponed for a first prototype and the questions still open. No prior familiarity with structure editors is assumed.

1 The Problem, and the Idea in Plain Terms

1.1 Why flat text is a limitation

Almost every programming language stores code as plain text: a file is a sequence of characters, read left-to-right, top-to-bottom. That format was inherited from early terminals and has barely changed since. It has a cost we rarely notice because we are so used to it:

- **Everything is always shown at once.** The code that runs the core algorithm, the logging that helps debugging, the type annotations, the visibility and mutability keywords — all of it sits on screen together, whether or not it is relevant to what you are currently trying to understand.
- **Mathematics is flattened.** A matrix, which is naturally a two-dimensional grid, must be written as $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$. A fraction must be written inline as a/b . The eye loses the alignment and shape that make such things readable on paper.

Both problems come from the same root: text is *one-dimensional* and *shows everything*. Mathematical notation, by contrast, has used two dimensions and selective emphasis for centuries, because doing so reduces mental effort.

1.2 The core idea

The proposal rests on a single shift. **Instead of storing code as text, store it as structure — the tree you would get by parsing it — and treat the on-screen text or diagram as just one possible *picture* of that structure.** Because the picture is generated from the structure rather than being the thing itself, we are free to draw the same code in different ways for different purposes.

Two kinds of adjustment follow naturally, and they are the heart of the system:

1. **Hiding detail by category.** The reader can say “hide all logging” or “hide all mutability keywords,” and every matching piece disappears from the picture at once — while remaining fully present in the program.
2. **Laying things out in two dimensions.** A matrix can be drawn as an aligned grid; a fraction as a numerator stacked over a denominator; even an `if`-statement can be drawn as two side-by-side columns. Again, only the picture changes.

A useful mental anchor: most IDEs already let you “fold” (collapse) a function or a comment block to tidy the screen. This proposal is that idea taken much further and made principled — hiding by *meaning* rather than by line range, drawing in two dimensions, and (importantly) guaranteeing that these adjustments never change what the program does.

A note on terminology. An editor that stores code as structure and shows adjustable pictures of it is known in the research literature as a *projectional* or *structure* editor (the picture is a “projection” of the underlying structure). We avoid the jargon in the body of this paper and simply speak of *structure* (what is stored) and *views* (how it is shown). The design here is opinionated about *which* adjustments to allow, and that discipline is the substance of the document.

1.3 A running example

Consider a small function that computes a weighted sum of two vectors and logs its progress. Written as ordinary text it might read:

```
fn combine(a, b):  
    log.debug("combine", a, b)  
    return 0.7 * a + 0.3 * b
```

In the proposed environment, the *same stored program* could be shown to a reader who has turned off logging as:

```
fn combine(a, b):  
    return 0.7 * a + 0.3 * b
```

with the weighted sum optionally drawn as a bordered chain of `+` and `*` operations, and `a` and `b` drawn as column vectors. Nothing about the program changed; only the reader’s chosen view did. We return to this example when discussing individual features.

2 The One Foundational Commitment

2.1 Structure is stored; appearance is generated

Everything in this proposal depends on one decision: **the source of truth is the structure, not any text file**. The program lives as a collection of parsed definitions; any text or diagram you see is produced *from* that structure on demand. This is what makes “hide all logging” or “draw this as a grid” possible at all — the system can act on categories and shapes because it knows what every part of the code *is*, not merely how it was typed.

Two consequences follow, both intentional:

- **Code is kept in a database of independent definitions, not in files.** Each function and each type is defined on its own. There is **no ordering** among top-level definitions — no notion of “this function must appear before that one,” no header files, no forward declarations. The system resolves references by identity.

- **The same program has many valid appearances.** There is no single canonical text. Appearance is always a choice made by the person reading, called a *view*.

Guiding premise. A program’s meaning is stored once, as structure, and has no built-in appearance. A *reader* selects a *view* — a bundle of hide/show choices and layout choices — that changes only how the program is drawn. Views are reversible, can be toggled while you work, and can be named, saved, and shared.

2.2 What “no ordering” does and does not mean

To avoid a common misreading: ordering is removed only among *top-level definitions* (the set of functions and types). Order that carries meaning *inside* a definition is untouched — the steps in a function still run in sequence, the arms of a pattern-match are still tried in order, the fields of a record still have their order. “No ordering” is a statement about the catalogue of definitions, not about control flow.

3 The Rules That Keep the System Honest

An environment this flexible could easily become confusing or misleading. The following rules are the safeguards. Every proposed feature is admitted only if it obeys them; several attractive-sounding features were rejected precisely because they do not.

3.1 Rule 1 — Separate “changing the view” from “changing the program”

This is the most important rule. Every property of a construct falls into exactly one of two kinds, and **the two are never altered by the same action**:

- **View properties** affect only appearance and can be freely flipped by the reader. Flipping one *cannot change what the program computes*. Example: whether an `if`-statement is drawn as two columns or stacked top-to-bottom; whether logging is shown.
- **Meaning properties** are part of the program. Changing one *is* editing the program, and produces a different result. Example: whether a vector is a row or a column — because a row vector V and a column vector V^T are genuinely different mathematical objects.

The danger to guard against: a single gesture (say, “rotate this”) that means “change my view” in one place but “change the value” in another. If a reader could accidentally transpose a vector — and thereby change the program’s results — while merely trying to adjust their display, trust in the whole system would collapse. This matters especially in science, where a silent change to a computation can quietly invalidate results.

No action crosses the line. View controls act only on view properties and can never alter results. Where a visual change *would* change meaning (such as transposing a vector), it is a separate *edit*, visibly marked in the code, and is never bound to the same control as a view adjustment.

3.2 Rule 2 — Borrowed notation is safe; invented notation must announce itself

Some two-dimensional forms are universally understood: a fraction bar, a matrix grid. These are *borrowed* from established mathematical convention; they identify themselves and need no explanation. Other layouts are *invented* by this system — for instance, drawing an `if`-statement

as two side-by-side columns, which has no standard convention. An invented layout **must carry a visible label or frame** that says what it is, because nothing about the arrangement alone tells the reader “these two columns are the two branches of an `if`.”

3.3 Rule 3 — A layout is allowed only if it is determined by the data or is a safe, reversible view

Two sources of layout are acceptable; a third is not.

- **Determined by the data.** The appearance is fixed by the content itself, with nothing to choose. A matrix entry simply sits at its row and column; there are no free positions to decide.
- **A reversible view.** The reader may flip between two layouts (e.g. `if` as columns vs. stacked) only if both forms can display *any* content and neither loses information. Then flipping is always safe.
- **Not allowed: automatic layout that shifts as you edit.** Arrangements the system computes on its own — which move things around as the code changes — are disorienting and are excluded for reader-facing constructs. (One deliberate exception exists: a graph whose layout the *author* fixes by hand; see §4.7.)

3.4 Rule 4 — Every visual device must reduce effort, not add to it

The point of the system is to lower mental load. Frames, labels, and markers are justified only where they genuinely aid reading. Where the surrounding context already makes something clear, such devices should shrink to mere spacing and alignment rather than pile up as boxes inside boxes.

3.5 Rule 5 — Hiding must be honest

When something is hidden, it should generally leave a small, visible trace — a marker that says “there is more here.” Otherwise the system reproduces a well-known spreadsheet hazard: a hidden column with a stray formula that everyone forgets about, leading to wrong conclusions. The exact form of this trace (which we call the hidden item’s *residue*) is still an open design question (§6) and is identified as the single biggest factor in whether the system is trusted.

4 The Features, and the Limits on Each

4.1 Hiding detail by category

Ordinary IDE folding hides a *range of lines*. This system hides by *what something is*, everywhere it occurs, at once:

- Turn off “all logging,” “all mutability keywords,” “all visibility keywords,” or “all type annotations,” and every instance disappears from the view — including small in-line markers such as a `mut` inside a declaration, which line-based folding cannot target.
- A particular combination of what-is-hidden is itself a **named, saveable, shareable** thing — for example a “reading view,” a “review view,” and a “debugging view.” These live with the reader, never baked into the program.
- The menu of hideable categories starts **small and fixed** — a deliberate statement about which kinds of clutter matter most. Letting users define their own categories is postponed.

When notation is allowed to change how code looks. A construct earns a special appearance only when that appearance lowers effort *without* creating a second, competing way to express the same computation. Layout may change how existing structure is *displayed*; it may never add new structure or a new way to compute.

4.2 Textbook layout for mathematical objects

Restricted to a few well-understood forms, deliberately not attempting to cover all of mathematical notation:

- **Fractions** are drawn stacked (numerator over denominator).
- **Vectors, matrices, and small (up to three-dimensional) tensors** are drawn as aligned grids; a three-dimensional tensor as a short stack of matrix “slices.” Beyond three dimensions there is no helpful notation, so those are excluded.
- Grid rendering **aligns numbers automatically** (for example, on the decimal point), restoring readability that inline text cannot preserve.
- Deliberately *omitted*: summation (Σ) and product (Π) notation. The language already has higher-order functions (**fold/reduce**) that express these; adding a second notation for the same thing would only create confusion about which to use.

4.3 Two-dimensional layout for ordinary code constructs

- An **if**-statement may be shown with its two branches as **side-by-side columns** or **stacked** (the “then” part above the “else” part), and the reader can flip between these live.
- Because this layout is invented rather than borrowed, the construct **carries a label** identifying it as an **if** and marking which part is the condition (per Rule 2).
- Only layouts that can display *any* branch content and lose no information are offered, so flipping is always safe (per Rule 3). Layouts that look good only when a branch is short are excluded.
- Which layout is shown is a **reader’s choice**, not something the author fixes and not something the system decides automatically.

4.4 Keeping your place when a view changes

Because a reader can flip layouts while editing, the system must not lose the cursor. **The cursor marks a position in the structure, not a spot on the screen.** When a layout flips, the cursor stays on the *same piece of code* and its screen position is recomputed. Flipping should feel like turning an object you are holding — your grip stays on the same part. (This is only possible because the code is stored as structure; in plain text, “the same position” after a reflow is not well defined.)

4.5 Operator chains

- A *chain* is a single operation with *many* inputs — for instance “the sum of these four things” — stored as one node with four children, rather than as nested pairwise additions. This matches the idea of a **fold/reduce** over its inputs.
- It is drawn as a row (or column) of infix operators, e.g. $a + b + c$, wrapped in a **thin border** that shows exactly where the single chain begins and ends — useful especially when chains are nested inside one another.

- **Restricted to associative operators**, and initially to just $+$ and $*$. Associativity (the property that grouping does not matter, so $(a+b)+c$ equals $a+(b+c)$) is what makes it safe to show the operation as a flat list with no grouping. Non-associative operators such as subtraction and division are *not* made safe by the border alone — their grouping carries meaning — and are postponed.
- Chain **orientation** (drawn as a row or a column) is a view property only, since the result of $+$ or $*$ does not depend on it.
- **Editing behavior:** typing the chain’s operator next to an existing chain of the same operator **extends that chain** (adds another input) rather than building a new nested operation. A chain’s inputs may themselves be complex, two-dimensional things (a fraction, a matrix), so a chain must also cope with tall contents.

4.6 Vectors, and why their orientation is special

- A vector drawn as a row versus a column is not a cosmetic choice: a row vector and a column vector are **different values** ($V \neq V^T$). Orientation here is a *meaning* property, not a view property.
- A vector therefore **carries a marker** announcing that its orientation is meaningful — so a reader can distinguish it from, say, a $+$ -chain that merely happens to be drawn vertically, and knows that flipping it is not a free view change.
- The reader’s ordinary orientation control **does nothing** to a vector (ideally with a brief explanation the first few times, to teach the distinction).
- Turning a vector from a column into a row — transposing it — is a **separate editing action** that visibly changes the value (for example, writing V^T). It is never the same gesture as a view flip.
- Adjusting a chain’s orientation **does not propagate** into the vectors inside it: flipping how a sum of vectors is laid out rearranges the sum only, and leaves each vector’s own orientation untouched.

4.7 Hand-drawn graph constants

This is the one deliberate exception to “appearance is never authored,” admitted because it earns its place.

- **Motivation.** Code often defines a small graph — for example, a dependency graph of tasks — where the *spatial arrangement itself carries meaning* (this task is upstream; these run in parallel; that is where they join) that a bare list of edges does not convey.
- Node positions are **placed by hand by the author** and are treated as part of the program’s content. They are **preserved** through any hiding or layout changes elsewhere. The system does *not* rearrange the graph automatically.
- Because its appearance is authored content, the graph follows different rules from everything else and is treated as its own separate category.
- Editing a graph is a **drag-the-nodes, direct-manipulation task** — a different style of interaction from the keyboard-driven structural editing used everywhere else, and a recognized rough edge.
- **Excluded:** graphs the *system* lays out automatically. Automatic graph drawing has no single “correct” picture, and its layout would shift unpredictably as the code changes —

violating Rule 3. Where a graph's data alone should determine its appearance, it is better shown as an **adjacency matrix** or an aligned **edge table** (both of which come for free from the grid feature). Restricting to *planar* graphs (those drawable without crossing edges) was considered and rejected: planarity yields a tidy picture but still no single canonical one, and the system depends on appearances being either data-determined or safely reversible.

5 Decisions Deliberately Postponed for a First Prototype

The first thing to build is a **small prototype whose only job is to test the interaction ideas**, not to settle the final architecture. Where the “proper” choice and the “simplest” choice differ, the prototype takes the simplest and revisits later. The following are postponed on purpose.

- **How the code is actually stored.** Whether the final store is a genuine database or text-with-generated-views is left open. For the prototype, definitions can simply be held in memory (a lookup from name to parsed form) and loaded from ordinary files at start-up — giving the same observable behavior (no ordering, no user-visible files) without building real storage.
- **How definitions are identified.** Whether a function is referred to by name or by a fingerprint of its contents (a technique called *content addressing*, which makes renaming cheap and detects identical definitions) is postponed. The prototype uses plain **names** and a simple lookup table — easy to inspect, and enough to test the interaction ideas.
- **Where a reader’s view settings are stored** (per user, per project, or per file) is an implementation detail to decide later.
- **How safe hiding is guaranteed.** For now, hideable code is simply **tagged as hideable** by the author, and the tool trusts the tag without checking it. A stronger version — in which the compiler *proves* that hidden code cannot affect results before allowing it to be hidden — is postponed but noted as a known follow-up. The prototype tests the weaker, trust-the-tag version.
- **Non-associative operator chains** (subtraction, division) are out of initial scope; only **+** and ***** chains are built first.
- **Hiding individual nodes within a graph.** In the prototype a graph is **all-or-nothing**: single nodes cannot be hidden (though the whole graph may collapse as a unit). This sidesteps thorny questions about what happens to hand-placed positions when some nodes vanish. As a result the prototype tests only the weaker claim that hand-drawn graphs are worth having and pleasant to make and read; the more compelling use — letting a reader view just a sub-part, such as a critical path — is postponed.
- **Collaboration, comparison, and merging.** By storing code as structure rather than text, the system opts out of ordinary line-by-line **diff** and merge tooling; a replacement is genuinely hard and is postponed (single-user for the prototype). The hand-drawn graph is noted as the one feature that makes future merging harder, since it is the only place where *appearance* is content and two people could disagree about node positions with no difference in meaning.
- **Compact single-character symbols** for attributes such as mutability and visibility were an early idea and are now **set aside**: denser symbols give the reader *more* to decode, which works against the goal of reducing effort. The stronger move is to *hide* such an annotation when it is not needed, rather than to compress it into a clever glyph.
- **What to build it on** is left open while the focus is on the product concept; a display technology with flexible layout (such as a web browser’s rendering) is the natural expectation once implementation begins.

6 Open Questions

1. **What a hidden item leaves behind (its residue).** Nothing at all? A small dot, a caret, a faint tint, a margin mark? And how should this differ across categories (logging vs. mutability vs. visibility) and across layouts (a hidden **else** branch shown as a column vs. stacked)? This is judged the central question, because it determines whether readers *trust*

the hiding.

2. **How the adjustments combine.** When several features overlap — a bordered chain containing a matrix, sitting inside a labelled `if` column, with something hidden nearby — does the result read as clean, nested structure or as a clutter of boxes within boxes? Keeping it calm (letting frames shrink to spacing where context suffices) is the hardest and most important part to get right, and is where prototyping effort should concentrate.
3. **Whether editing feels fast.** Editors that store code as structure have historically been admired but little used, usually because *typing and editing* in them felt slow and clumsy compared to text. The prototype must test *editing*, not just viewing: does extending a chain feel natural; does editing a grid (tabbing between cells, adding rows, and especially *past*ing a block of numbers straight into a matrix) feel fluent; does the deliberate split between “flip the view” and “transpose the value” reliably prevent accidental changes?
4. **How much the reader types vs. manipulates.** Whether the user mostly types normal text that the system keeps parsed into structure, or instead manipulates structural pieces directly with little or no typing, is unresolved. The second is the “purest” form of the idea but carries the ergonomic risk above.
5. **Who the first user is, and whether this is a language or a layer.** Which scientific-computing audience to serve first — researchers who think in mathematical notation, or maintainers of large shared codebases — and, relatedly, whether the offering is a brand-new language or instead an editing-and-comprehension *layer* over an existing one. Most of the proposed value is about *interacting with* code rather than about language semantics, which makes the “layer” option worth serious consideration, given how hard it is to displace established languages and their libraries.
6. **Whether authors may suggest a default view.** The strict “the reader controls all appearance” stance costs the author the ability to say “this `if` is meant to be read side-by-side, because comparing the branches is the point.” Whether to allow a gentle author-suggested default that the reader can still override is left open.
7. **What a chain’s border conveys.** Whether the border shows only the chain’s extent (relying on the visible operators to say which operation it is) or also carries the operator’s identity (which becomes necessary if a chain can ever be collapsed to hide its contents).

7 Summary

The proposal is a **single medium for writing and reading scientific code in which the program is stored as structure and its on-screen appearance is freely adjustable by the reader**. One commitment underpins it — store meaning once, generate appearance on demand — and one discipline protects it: cleanly separate *changing the view* from *changing the program*, so that adjusting how code looks can never alter what it does. Around this sit a small, carefully limited set of features: hiding clutter by category, with saveable named views; textbook layout for fractions and small arrays; labelled, reversible two-dimensional layouts for ordinary constructs such as `if`; many-input operator chains limited to associative operators and marked by a border; vectors whose orientation is meaningful and whose transpose is an explicit edit; and hand-drawn graph constants admitted as a separate, author-controlled exception. Each feature is allowed only where its appearance cannot mislead and where it reduces rather than adds mental effort. The immediate goal is a deliberately minimal prototype that tests the interaction ideas — above all the honesty of hidden-item markers, the way overlapping adjustments combine on screen, and whether editing in this medium feels fast — before any lasting architectural choice is fixed.